<?php

MATERIAL DE ESTUDIO / DESARROLLO WEB

UNIDAD DIDÁCTICA 02

Relación 01
Clases y POO

Práctica progresiva de aula · enunciados y verificaciones

Objetos | Composición | Herencia | Excepciones

APUNTES DE CLASE

Teoría · tutoriales · actividades · código PHP

{ PHP }

UD2 · DWES · PHP 8.1 o superior

Objetivos

- Modelar objetos con atributos privados y constructores.
- Separar clases y punto de entrada con `require_once` y `__DIR__`.
- Practicar composición, herencia, clases abstractas, interfaces y namespaces.
- Validar datos y comunicar errores mediante excepciones.

Norma común: queda prohibido utilizar `echo`, `print`, `print_r` o HTML dentro de clases, métodos o funciones auxiliares. Toda salida se realiza desde `index.php`. Se valoran nombres expresivos, control de errores y archivos separados.

Ejercicio 1. Préstamos de una biblioteca · Guiado · 35–45 min

Crear una clase `Libro` con ISBN, título, autor y estado de préstamo. El ISBN y el resto de atributos son privados. Implementar `prestar(): bool`, `devolver(): bool`, `estaDisponible(): bool` y `descripcion(): string`. Implementar `Biblioteca` que permita añadir libros, buscar por ISBN y obtener todos los disponibles. No permitir ISBN repetidos. Desde `index.php`, insertar tres libros, prestar el primero, intentar prestarlo de nuevo, devolverlo y listar disponibles.

Comprobación: el segundo préstamo no modifica el estado; un ISBN duplicado produce `InvalidArgumentException`.

Ejercicio 2. Garaje y composición de objetos · Semiguiado · 45–60 min

Crear `Ventana`, `Puerta` y `Coche`. Cada puerta **contiene su propia ventana**; el coche contiene un array de puertas (no deben compartir la misma instancia). Cada objeto debe poder abrirse/cerrarse. El coche permite `abrirPuerta(indice)`, `cerrarTodo()`, `numeroPuertasAbiertas()` y `resumen(): string`. Impedir índices inválidos con excepción.

Comprobación: abrir una ventana no abre las demás; `cerrarTodo` cierra puertas y ventanas.

Ejercicio 3. Envíos polimórficos · Autónomo · 45–60 min

Crear la clase abstracta `Envio`, con propiedad protegida `pesoKg` y método abstracto `calcularPrecio(): float`. Implementar `EnvioNormal` (4 € + 1,50 €/kg) y `EnvioExpress` (9 € + 2,75 €/kg). Crear interfaz `Rastreable` con método `codigoSeguimiento(): string`; solo `EnvioExpress` la implementa. Comprobar que el peso es estrictamente positivo. El punto de entrada muestra el precio de cada envío y el código de seguimiento si corresponde.

Comprobación: los precios difieren para igual peso, y el código imprime solo en el punto de entrada.

Ejercicio 4. Inventario con namespaces · Autónomo · 60–75 min

Crear `App\Dominio\Articulo` (código, nombre, stock) con `aumentarStock(cantidad)` y `retirarStock(cantidad)`. Para stock insuficiente, lanzar la excepción propia `App\Excepciones\StockInsuficienteException`, que hereda de `RuntimeException`. Organizar namespaces y `require_once` sin Composer. En `index.php`, ejecutar altas y retiradas y capturar la excepción sin que el estado quede negativo.

Comprobación: stock nunca negativo; el mensaje de error se presenta únicamente desde `index.php`.

Entrega

Cada ejercicio se entrega en su carpeta, con `index.php` y las clases separadas. Para ejecutarlos, usar `php index.php` (o navegador a través de XAMPP). No se requiere MySQL.